

GapHarness: Certificate-Carrying Runtime Harness Compilation for API-Only LLM Agents

Haocheng Lu
haocheng409@gmail.com

Abstract

API-only LLM agents increasingly depend on external runtime support: retrieval, code execution, workspace state, sandbox actions, permission gates, and verification. Existing tool routers usually select tools directly, conflating two questions that should be auditable separately: what obligations a user request imposes, and what runtime support can satisfy those obligations under a declared registry. We introduce GapHarness, a research system for obligation-first runtime harness compilation. Given an obligation profile and a finite registry of modules with declared affordances and design-time costs, GapHarness compiles the lowest declared-cost module subset that satisfies required obligations, capabilities, and dependencies, or explicitly returns unsupported. The compiler is exact within the declared registry and returns certificate-carrying outputs that expose coverage, dependency satisfaction, missing affordances, and registry-relative minimality evidence.

We evaluate GapHarness on GapBench-1000, a controlled project-owner-audited benchmark for obligation/capability coverage, plus targeted boundary diagnostics and executable sandbox traces. Under gold profiles, GapHarness exactly matches the oracle minimal harness on GapBench-1000; this is compiler validation, not the main performance claim. With LLM-inferred profiles on held-out test800, GapHarness reaches 0.89 harness success, compared with 0.80 for a strong LLM Tool Router that sees the same registry and declared costs but not obligation labels. Feedback-assisted baselines sharpen the claim: verifier-repair and ReAct-style selectors can reach 1.00 harness success when given verifier diagnostics after failed routes, sometimes at lower declared cost, but they do not produce minimality certificates. GapHarness-Repair converts the same kind of diagnostics into profile patches and recompiles through the exact compiler, reaching 1.00 harness success on test800 and HarnessChallenge-200 while preserving certificates. We add feedback-level replay, certificate-utility proxies, cost-scheme sensitivity, status confusion matrices, profiler error taxonomies, registry perturbation, label permutation, negative controls, and SWE-HarnessExec executable traces. These results support a bounded systems claim: obligation-first compilation separates profile inference from certified registry-constrained support selection, improving auditability and certificate availability without claiming raw-cost dominance over iterative repair. It does not measure open-world answer correctness, SWE-bench pass@1, or dominance over arbitrary agent frameworks.

1. Introduction

Modern LLM agents are often evaluated as end-to-end systems. A model receives a request, selects tools, acts in an environment, and returns an answer. This framing is practical, but it hides a narrower engineering problem that appears in almost every API-only agent stack: before acting, the system must decide what external runtime support is required.

Consider a user request: “Using the files in this workspace, run the tests, patch only the sandbox copy, and tell me whether the fix passes; do not touch production.” A direct tool router may select a code executor but miss the workspace-state reader, sandbox editor, permission gate, or trace verifier. An always-full harness may include all support but pay avoidable cost and blur the safety boundary. GapHarness instead infers Observation, Execution, State, Action, Control, and Verification obligations, then compiles the minimal

declared support set, or returns unsupported if sandbox editing or permission gating is absent.

A prompt may require observing evidence beyond the prompt, executing deterministic code, inspecting a workspace, maintaining durable state, editing a sandbox artifact, applying a permission gate, or verifying a contract. These are not merely tools. They are obligations imposed by the request if the answer or action is to be warranted. Tool-using agents such as ReAct, Toolformer, Gorilla, ToolLLM, and MetaTool study when and how models use tools (S. Yao et al. 2023; Schick et al. 2023; Patil et al. 2023; Qin et al. 2024; Huang et al. 2023). GapHarness studies a different layer: given a profile of obligations and a declared module registry, what is the minimal harness that covers those obligations?

This separation matters for reliability. If a task requires an affordance absent from the registry, a system should return unsupported or under-covered rather than silently pretending to support it. If a query explicitly says not to use tools, a system should not over-harness merely because it contains words such as “search”, “code”, or “file”. If a profiler corrupts obligation labels, the compiler/verifier stack should fail measurably instead of passing arbitrary labels.

This paper makes a controlled systems claim. GapHarness is not a general-purpose agent framework and does not solve GAIA, Terminal-Bench, or arbitrary real-world side-effect safety. It provides a minimal, auditable runtime-support compilation layer for API-only agents under a declared ontology, registry, dependency model, and cost function.

Contributions.

1. **Abstraction.** We formulate API-only harnessing as obligation inference followed by finite declared-registry compilation.
2. **System.** We implement an exact certificate-carrying compiler that returns a lowest-cost valid module subset, direct/clarify/unsupported status, and checkable coverage, dependency, and minimality evidence.
3. **Evaluation.** We evaluate coverage, cost, auditability, feedback dependence, registry boundaries, label sensitivity, and executable trace behavior against direct routers, LLM routers, workflow generation, verifier-repair, and ReAct-style selection under a shared registry.

2. Related Work

Agent tool use and workflow generation. Tool-use research focuses on models that decide when or how to call external APIs. ReAct interleaves reasoning and acting (S. Yao et al. 2023), Toolformer trains API-use behavior (Schick et al. 2023), Gorilla connects LLMs with large API collections (Patil et al. 2023), and ToolLLM/ToolBench scale instruction tuning and evaluation for API use (Qin et al. 2024). MetaTool directly evaluates tool-use necessity and tool selection (Huang et al. 2023). AutoFlow, AFlow, and WorFBench optimize or benchmark agentic workflow generation (Li et al. 2024; Zhang et al. 2025; Qiao et al. 2025). GapHarness differs by inserting an obligation layer before module selection. The compiler does not infer that a request needs “search”; it checks whether the profile imposes Observation, Execution, State, Action, Control, or Verification obligations and then compiles declared modules that cover them.

Harness engineering and execution benchmarks. AutoHarness synthesizes code harnesses around agents (Lou et al. 2026), Natural-Language Agent Harnesses study harness specification (Pan et al. 2026), and Harness-Bench measures harness effects across model/harness configurations (Y. Yao et al. 2026). Agent benchmarks such as AgentBench, GAIA, Terminal-Bench, WildToolBench, and MCP-Bench evaluate richer interactive or tool-use settings (Liu et al. 2024; Mialon et al. 2024; Merrill et al. 2026; Yu et al. 2026; Wang et al. 2025). GapHarness uses such artifacts only as external-validity and boundary diagnostics unless answer-level execution and task-specific grading are implemented.

Configuration, composition, and planning. The compiler resembles classical weighted set cover and constrained service/workflow composition: modules cover capabilities at costs, and an exact optimizer searches for a feasible minimum (Chvátal 1979; Karp 1972; Rao and Su 2005). Feature-model configuration and software product-line analysis also study valid selections under constraints (Kang et al. 1990; Benavides, Segura, and Ruiz-Cortés 2010). GapHarness is not a new approximation algorithm for set cover. Its contribution is

the certificate-carrying runtime harness compilation abstraction for LLM agents, with obligation inference separated from registry-constrained support selection.

Runtime assurance and policy. Runtime verification, policy-as-code, capability-based security, self-adaptive MAPE-K loops, and assurance cases provide vocabulary for checking behavior against explicit runtime claims (Leucker and Schallhart 2009; Saltzer and Schroeder 1975; Kephart and Chess 2003; Kelly and Weaver 2004). GapHarness is closer to this assurance layer than to open-world tool-use accuracy: it asks whether a selected runtime support set is declared, sufficient, minimal under cost, and auditable. We therefore compare harness-selection strategies implemented over the same registry, model, executor, and verifier rather than claiming that GapHarness is categorically better than framework substrates such as LangGraph, AutoGen, or an Agents SDK. Those frameworks can implement many policies, including GapHarness itself.

3. Problem Formulation

Let $\mathcal{O}$ be a finite set of obligations and $\mathcal{C}$ a finite set of lower-level capabilities. In this work,

$$\mathcal{O} = \{\text{Observation, Execution, State, Action, Control, Verification}\}.$$

A task profile is

$$p = (O_p, C_p, s_p, r_p),$$

where $O_p \subseteq \mathcal{O}$ is the required obligation set, $C_p \subseteq \mathcal{C}$ is the required capability set, s_p is one of supported, unsupported, or clarify, and r_p is risk or output-contract metadata.

A registry R is a finite set of modules. Each module $m \in R$ declares obligations O_m , capabilities C_m , dependency requirements D_m , and a non-negative cost $w_m \geq 0$. For a selected subset $S \subseteq R$,

$$O(S) = \bigcup_{m \in S} O_m, \quad C(S) = \bigcup_{m \in S} C_m, \quad W(S) = \sum_{m \in S} w_m.$$

We say that S is valid for a supported profile p when

$$O_p \subseteq O(S), \quad C_p \subseteq C(S),$$

and all dependency predicates declared by modules in S are satisfied by $O(S)$ and $C(S)$. For clarification profiles, the compiler returns clarify. For unsupported profiles, the compiler returns unsupported unless the registry explicitly supports the required real-world side effect.

Proposition 1 (Relative minimality). Given a finite registry R , non-negative module costs, and exact subset search, if there exists a subset $S \subseteq R$ that is valid for profile p , GapHarness returns a valid subset with minimum cost among all valid subsets. If no valid subset exists, it returns unsupported.

Proof sketch. GapHarness enumerates all subsets of the finite registry, filters out subsets that do not cover O_p , C_p , or module dependencies, and selects the remaining subset with minimum $W(S)$. Because the candidate set is finite and costs are non-negative real numbers, a minimum exists whenever the valid set is nonempty. If the valid set is empty, the compiler has no covering subset and returns unsupported. The result is relative to the given registry, dependencies, and cost function, not an absolute optimality claim.

Proposition 2 (Declared-boundary failure). If a supported profile requires a capability $c \in C_p$ and no dependency-satisfying subset of R provides c , exact GapHarness compilation cannot return a valid supported harness.

Proof sketch. For every subset $S \subseteq R$, $c \notin C(S)$ by assumption, so $C_p \not\subseteq C(S)$. Thus the valid subset set is empty and Proposition 1 implies unsupported. This is the formal reason registry perturbation should degrade into unsupported, under-covered, or verifier-fail status rather than silent success.

Proposition 3 (Verifier-visible label corruption). If a corrupted profile p' causes the compiler to select a harness S' that does not cover the original audited profile p , then a verifier checking against p must fail coverage.

Proof sketch. The verifier checks $O_p \subseteq O(S')$ and $C_p \subseteq C(S')$. If either inclusion is false, it emits a missing-obligation or missing-capability failure. The claim is conditional: some corruptions can be harmless if they compile to the same covering harness, but arbitrary labels cannot be guaranteed to pass.

We also define a module dominance relation used by the optimized compiler. The registry model is monotone: modules add declared obligations/capabilities and dependency requirements, and they do not encode negative conflicts or mutual exclusion. Under this positive-coverage model, module a dominates module b if $O_a \supseteq O_b$, $C_a \supseteq C_b$, $w_a \leq w_b$, and the dependency requirements of a are no stricter than those of b . The implementation uses a tie-safe version of this rule so that deterministic tie-breaking remains identical to brute-force exact search.

Proposition 4 (Dominance pruning preserves optimality). Removing dominated modules under the tie-safe dominance rule preserves the registry-relative minimum-cost harness selected by exact search.

Proof sketch. For any valid subset containing a dominated module b , replacing b with its dominator a preserves obligation coverage, capability coverage, and dependency satisfiability because a covers a superset with no stricter dependencies. The replacement does not increase cost. Under the tie-safe condition it also cannot lose the deterministic tie-breaker used by the brute-force compiler. Therefore no optimal selected harness is removed by dominance pruning.

4. System

GapHarness has five components.

The profiler maps a query to an obligation profile. Experiments use gold profiles, heuristic profiles, LLM-inferred profiles, and a post-hoc registry-guarded LLM profile variant.

The registry declares available modules, their provided obligations/capabilities, dependencies, costs, and verifier metadata. The MVP registry contains web retrieval, source-span checking, Python execution, execution-log checking, workspace inspection, durable state storage, sandbox file editing, permission gating, contract verification, and trace recording.

The compiler is a certificate-carrying exact optimizer over the declared registry. It first normalizes the profile against the registry vocabulary. Safety and evidence closure constraints are encoded as required capabilities and module dependencies, e.g., sandbox mutation requires permission, execution verification requires an execution log, and source-backed observation requires source-span checking. The compiler then removes tie-safe dominated modules, runs branch-and-bound exact search, and returns direct-answer, supported, clarify, or unsupported status.

The search remains exact. A branch is pruned only when its current cost already exceeds the best known valid harness, when the selected modules plus all remaining modules cannot cover the missing obligations/capabilities, or when a valid partial harness has already been found and adding more non-negative-cost modules cannot improve it. The output includes a deterministic, checkable certificate. A coverage certificate records selected modules, covered obligations/capabilities, missing affordances, and total cost. A dependency certificate records whether every selected module’s declared prerequisites are satisfied. A minimality certificate records search statistics and, when the registry is small enough to enumerate, lower-cost invalid candidates; under branch-and-bound it certifies that the exact search procedure found no lower-cost valid subset. This verifier does not call the LLM.

```

Compile(profile p, registry R):
  if p requires clarification: return clarify certificate
  if p is direct-answer sufficient: return empty harness certificate
  R' = remove tie-safe dominated modules from R
  best = infinity
  DFS(partial S, remaining modules):
    prune if cost(S) > cost(best)
    prune if S plus remaining cannot cover O_p or C_p
    if S is valid: update best and stop this branch
    branch include/exclude next module
  if best exists: return best with coverage/minimality certificate
  else: return unsupported with missing-affordance certificate

```

The default executor is deterministic and sandboxed. It records traces and mock actions but does not perform irreversible external side effects. For executable validation, we add a separate sandbox fixture runner that creates local repositories, runs pytest before and after a provided patch, records logs, and verifies the trace contract.

The verifier checks expected status, obligation/capability coverage, dependency constraints, and drop-one minimality diagnostics.

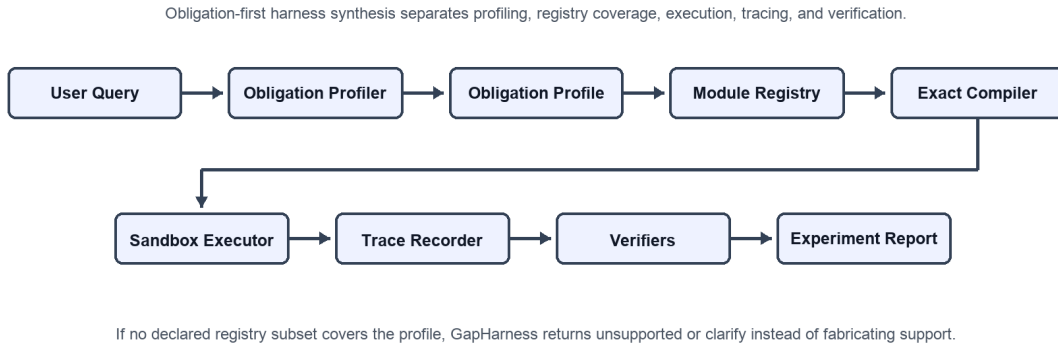


Figure 1: GapHarness pipeline.

5. Benchmarks and Metrics

GapBench. GapBench v1.0 contains 1000 controlled tasks with project-owner-audited obligation labels, capability labels, expected status, oracle minimal harness, risk metadata, and provenance. It has dev200 and test800 splits. GapBench is controlled and factorial by design. Its purpose is to isolate harness compilation and coverage failures, not to measure general assistant quality.

Targeted diagnostic benchmark. HarnessChallenge-200 is a deliberately constructed diagnostic suite, not a natural-frequency benchmark. It contains minimal pairs, hard tool-bait, sandbox/mock versus real side-effect boundaries, absent registry affordances, evidence/verification traps, and real-source paraphrases. Its purpose is to test whether harness compilation is obligation-sensitive and registry-boundary-sensitive under adversarially chosen prompts.

Fresh boundary holdout. RealBoundary-100 is an author-seeded boundary holdout constructed after the post-hoc registry guard was fixed. It targets sandbox/mock/local versus real-world side-effect distinctions. It is not used to tune the guard, but it is also not yet independently human-audited; we therefore report it as a fresh boundary diagnostic and provide a review sheet rather than treating it as primary evidence.

Independent naturalistic holdout package. Naturalistic-Holdout v0.1 is a candidate review package containing 200 public GitHub issue-derived agent-development and developer-tooling requests collected through the unauthenticated GitHub REST API. It is independent of GapBench: it is not generated from GapBench templates and is not a naturalization of GapBench rows. The package includes provenance, redacted snippets, a review sheet with Observation/Execution/State/Action/Control/Verification, capability, expected-status, annotator, and adjudication fields, and a two-annotator plan for Cohen’s kappa or Krippendorff’s alpha plus disagreement adjudication. We do not report scores on it because the rows are candidates, not human-audited gold.

Executable trace validation. SWE-HarnessExec-20 contains 20 sandboxed software-maintenance fixtures, and SWE-HarnessExec-50 extends the deterministic provided-patch trace check to 50 fixtures. Each fixture creates a local Python repository, starts with a failing pytest test, applies a provided patch to `solution.py`, reruns pytest, and verifies that the trace includes inspection, execution logs, sandbox editing, state, permission, and contract verification. These are executable trace validations; they are not SWE-bench checkouts, model patch generation, or SWE-bench pass@1.

External-validity and boundary diagnostics. GAIA-Transfer contains 200 GAIA-derived obligation-transfer rows. It checks whether the representation and compiler can process transfer-style labels, not whether the system solves GAIA answer-level tasks. GapBench-Natural contains 200 project-owner-audited naturalized prompts derived from GapBench source rows. SWE-Obligation-50 contains 50 project-owner-audited obligation-transfer rows derived from public SWE-bench Lite issue/task descriptions and test metadata; it is not repository checkout, patch generation, or pass@1 evaluation. Terminal-Bench-obligation50 remains a terminal-style scaffold, not a Terminal-Bench solving result.

Harness success. Harness success is deterministic verifier pass against expected status and project-owner-audited obligation/capability coverage. It is not answer-level correctness.

The `success_checker` field in benchmark rows is provenance metadata describing the intended evaluation boundary, such as `gaia_obligation_transfer_only` or `swe_obligation_transfer_only`. The executable verifier used in the experiments checks expected status, obligations, capabilities, dependencies, and trace artifacts; it does not use `success_checker` names as separate hidden answer validators.

Cost metrics. Cost delta is the mean predicted harness cost minus the mean oracle minimal cost. It can be negative when insufficient harnesses are too cheap, so it should not be called non-negative regret. Excess cost is the mean per-task positive excess, $\mathbb{E}_q[\max(0, \text{cost}(q) - \text{oracle_cost}(q))]$. Therefore excess cost can be positive even when aggregate cost delta is negative.

Declared module costs are design-time costs, not measured provider prices. To make this explicit, we also report sensitivity under uniform, latency-proxy, token/API-proxy, risk-weighted, and random-perturbed costs.

Failure metrics. Over-harnessing means predicted cost exceeds oracle minimal cost on a supported task. Under-harnessing means a supported task fails coverage. Wrong-harnessing means a nonempty selected harness still has verifier failures. These rates are not mutually exclusive.

Uncertainty estimates. We report nonparametric bootstrap 95% confidence intervals over task rows for key rates and costs. The intervals are descriptive because several benchmarks are controlled or targeted diagnostics rather than samples from a natural population.

6. Experiments

6.1 Gold-profile compiler evaluation

Table 1 isolates the compiler by using audited gold profiles. GapHarness matches the oracle minimal harness exactly on GapBench-1000. Direct answering is cheap but under-covered. Always-full attains high harness success but over-harnesses nearly all supported tasks.

Table 1: GapBench-1000 gold-profile compiler result. HS is harness success; cost columns use declared module costs.

System	N	HS	Decl. cost	Delta	Excess	Over	Under/Wrong
Direct	1000	0.20	0.00	-3.67	0.00	0.00	0.74 / 0.00
Tool Router	1000	0.34	2.10	-1.57	0.27	0.11	0.60 / 0.42
Difficulty Router	1000	0.43	3.46	-0.21	1.47	0.28	0.51 / 0.16
Always-full	1000	0.94	16.00	12.33	12.33	0.94	0.00 / 0.00
GapHarness	1000	1.00	3.67	0.00	0.00	0.00	0.00 / 0.00
Oracle minimal	1000	1.00	3.67	0.00	0.00	0.00	0.00 / 0.00

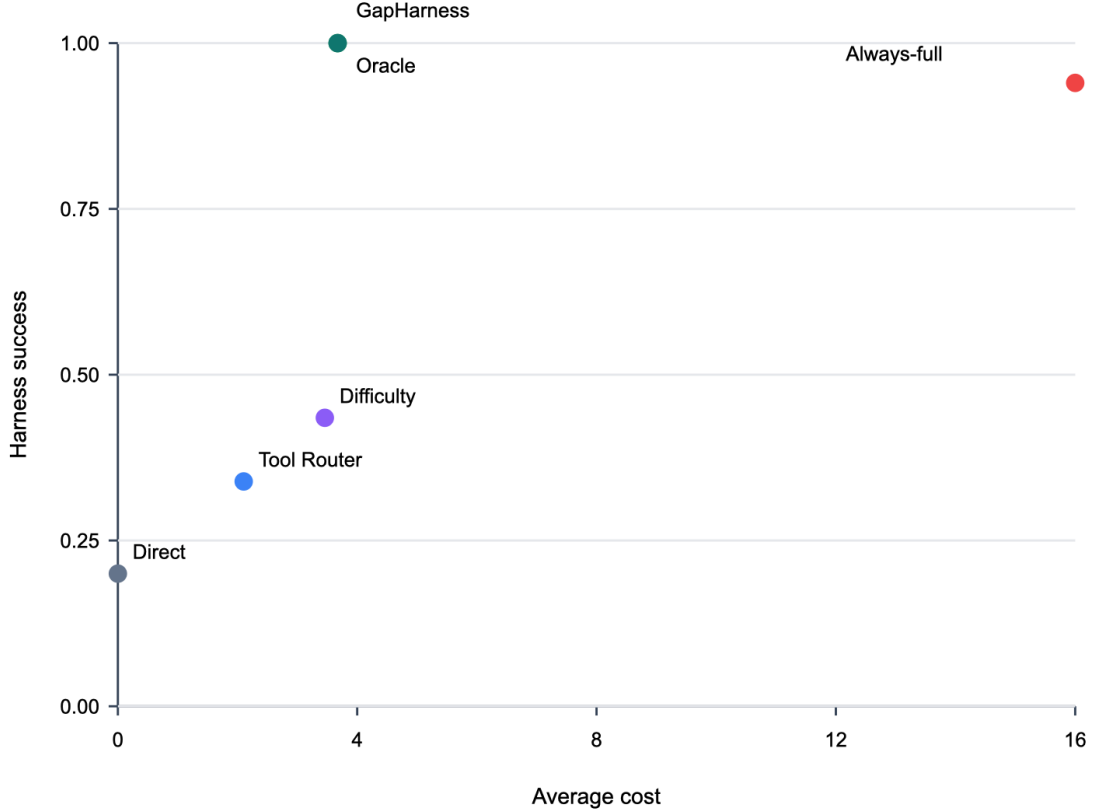


Figure 2: Declared-cost success frontier under gold profiles.

6.2 Compiler optimization, replay equivalence, and scaling

The optimized compiler is intended to change search runtime behavior and auditability, not harness semantics. We therefore replay frozen experiment rows through the new dominance-pruned branch-and-bound compiler while ignoring the newly added certificate metadata. Table 2 shows zero status, module, or declared-cost changes across 14,320 replay rows spanning GapBench-1000 gold, held-out LLM profiles, registry-guarded profiles, HarnessChallenge-200, LLM Tool Router routes, and SWE-HarnessExec-20. This supports the claim that the optimized compiler is extensionally equivalent to the previous exact compiler on all frozen profiles/routes.

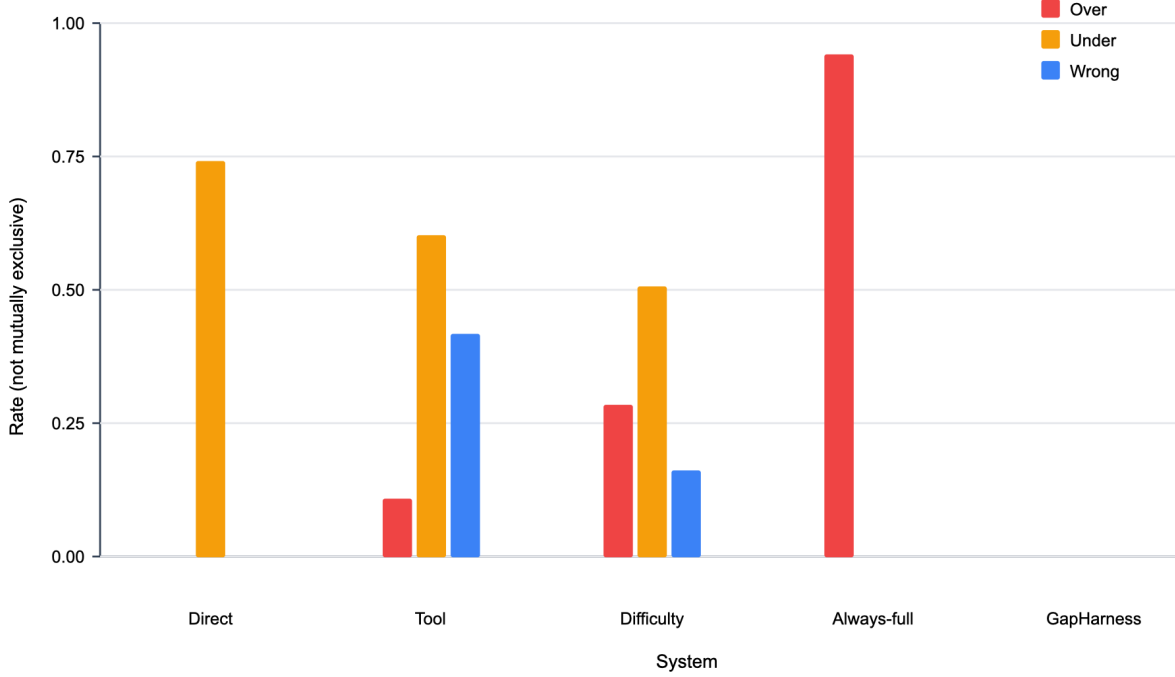


Figure 3: Grouped over/under/wrong rates on GapBench-1000. Rates are not mutually exclusive.

Table 2: Compiler equivalence replay. Certificates are new metadata and are ignored for equality.

Frozen experiment	N	Status changed	Harness changed	Cost changed
GapBench-1000 gold	6000	0	0	0
test800 LLM replay	5600	0	0	0
test800 registry-guarded	800	0	0	0
HarnessChallenge gold	1200	0	0	0
HarnessChallenge LLM/guarded/router	600	0	0	0
SWE-HarnessExec-20	120	0	0	0

We run two synthetic registry-scaling studies. The first is a dominated-redundancy setting: the registry contains a small set of useful affordance modules plus many strictly dominated redundant declarations. Naive brute force is run only up to 20 modules, where it evaluates 2^{20} candidates. The optimized compiler preserves the brute-force optimum on feasible sizes and compiles the 160-module dominated registry in under 1 ms while visiting 459 search nodes.

Table 3: Synthetic registry scaling. Brute force is skipped beyond 20 modules because exact enumeration is exponential.

Registry	Brute candidates	Brute ms	Opt ms	Opt nodes	Dominated	Greedy	Opt
10	1,024	1.77	0.68	459	1	9	9
20	1,048,576	2382.54	0.63	459	11	9	9
40	skipped	-	0.61	459	31	9	9
80	skipped	-	0.65	459	71	9	9
160	skipped	-	0.74	459	151	9	9

The second scaling study is intentionally less dominance-prunable. Modules overlap but are mostly non-dominated, so dominance pruning removes no modules. This setting documents the boundary of exact search rather than claiming polynomial scaling.

Table 4: Mostly non-dominated registry scaling stress. This is a boundary diagnostic for exact search, not a throughput claim.

Registry	Brute candidates	Brute ms	Opt ms	Opt nodes	Dominated	Greedy	Opt
20	1,048,576	2004.67	36.70	29,417	0	16	16
30	skipped	-	303.29	263,301	0	16	16
40	skipped	-	3297.16	3,003,289	0	16	16

The scaling result is not a polynomial-time claim. Worst-case exact compilation remains exponential, as the mostly non-dominated stress makes visible. The point is narrower: declared agent registries often contain redundant affordance declarations, and dominance/cost/coverage pruning can make exact, certificate-carrying compilation practical at the registry sizes considered while retaining explicit evidence about when the search becomes harder. We include 20 certificate samples covering supported, direct, unsupported, and perturbed-registry cases; each sample records selected modules, covered obligations/capabilities, dependency checks, missing affordances, search statistics, and low-cost invalid examples when enumerable.

6.3 LLM profiling and a fair LLM Tool Router baseline

We evaluate LLM-inferred profiles on held-out test800. To avoid an unfair comparison against only heuristic routers, we add an LLM Tool Router baseline. This baseline receives the module registry and costs and selects modules directly, but it is not shown the obligation ontology or gold labels. Its raw routes contain no named obligation labels.

On test800, the LLM Tool Router reaches 0.80 harness success. GapHarness with LLM-inferred obligations reaches 0.89, and the post-hoc registry-guarded variant reaches 0.94. The LLM Tool Router handles pure-language and tool-bait negative controls perfectly, but under-covers multi-obligation tasks. This is a useful outcome: a strong language model can avoid keyword-only over-harnessing, yet direct module routing remains weaker than obligation-first profiling on the main coverage task. The gap between LLM Tool Router and GapHarness suggests that asking the model to select modules directly is not equivalent to asking it to infer obligations first.

Table 5: Held-out test800 comparison. The LLM Tool Router sees registry modules and declared costs but not obligation labels.

System	N	HS	Decl. cost	Delta	Excess	Over	Under/Wrong
Direct	800	0.20	0.00	-3.69	0.00	0.00	0.74 / 0.00
Tool Router	800	0.32	1.96	-1.72	0.25	0.09	0.62 / 0.43
Difficulty Router	800	0.41	3.22	-0.47	1.34	0.26	0.53 / 0.15
Always-full	800	0.94	16.00	12.31	12.31	0.94	0.00 / 0.00
LLM Tool Router	800	0.80	3.51	-0.18	0.13	0.12	0.20 / 0.17
GapHarness LLM	800	0.89	3.59	-0.09	0.37	0.14	0.09 / 0.01
Registry-guarded GH	800	0.94	3.98	0.30	0.38	0.15	0.03 / 0.01
Gold oracle GH	800	1.00	3.69	0.00	0.00	0.00	0.00 / 0.00

On dev200, the same LLM Tool Router reaches 0.79 harness success, 3.35 average cost, -0.27 cost delta, 0.11 excess cost, 0.21 under-harnessing, and 0.20 wrong-harnessing. This confirms that the baseline is not a test-only artifact. A nonparametric bootstrap gives a 95% CI of [0.865, 0.910] for held-out GapHarness LLM harness success and [0.927, 0.960] for the registry-guarded variant.

6.4 Post-hoc registry-boundary calibration

The registry guard is a post-hoc registry-boundary calibration study, not a fresh held-out discovery. The initial LLM-profile sweep exposed a systematic unsupported false-positive pattern: sandbox/mock/local actions were sometimes lowered to unsupported real-world side effects. The guard preserves true external side-effect boundaries but removes `real_world_side_effect` when the query explicitly limits action to sandbox/mock/local scope.

On test800, the guard improves harness success from 0.89 to 0.94 and reduces unsupported false positives from 56 to 12. The tradeoff is a higher average cost and positive cost delta. This supports a narrow calibration claim about declared registry boundaries, not a general guarantee that every profiler error is fixed.

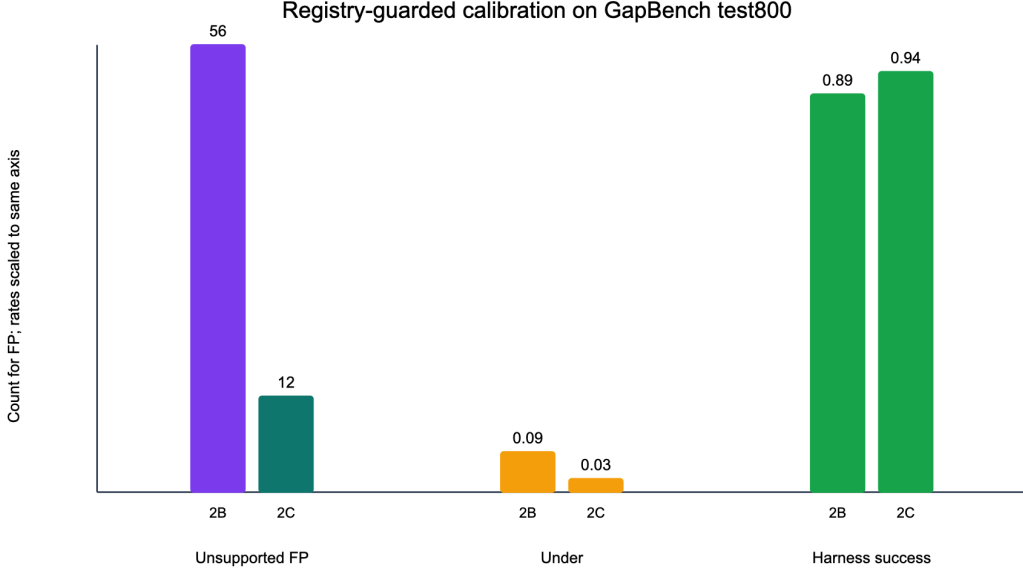


Figure 4: Post-hoc registry-boundary calibration on held-out test800.

To prevent this post-hoc calibration from becoming a hidden main claim, we separately create RealBoundary-100 after the guard was fixed and report it as a fresh author-seeded holdout. Under author-seeded profiles, GapHarness and the oracle minimal harness both reach 1.00 harness success at cost 4.20. Direct answering reaches 0.20, the deterministic tool router reaches 0.20, difficulty routing reaches 0.28, and Always-full reaches 0.60 because it over-supports supported rows but fails explicit unsupported side-effect boundaries. Since this holdout is not yet independently audited, it supports only a fresh boundary sanity check and a review protocol, not a final external-validity claim.

6.5 Diagnostic-feedback strategy baselines

Frameworks are execution substrates, not single algorithms. We therefore add stronger harness-selection strategies over the same declared registry, executor, verifier, model, and costs. A Workflow Generator directly emits a module workflow from the query and registry. A Verifier-Repair Router starts from the LLM Tool Router, receives verifier failures, and may add or remove modules for up to two repair rounds. A ReAct-style Module Selector iteratively selects modules, stopping, declaring unsupported, or clarifying within four steps; its first step receives no gold labels or verifier feedback, while later steps receive verifier feedback only after failed routes.

These feedback-assisted baselines are intentionally strong. After a failed route, verifier-repair and ReAct receive diagnostic feedback derived from the benchmark coverage checker, including missing capability/status information. They should be interpreted as diagnostic-feedback upper bounds, not as ordinary black-box ReAct or workflow baselines.

We also add GapHarness-Repair, a verifier-guided recompile variant. It starts from the one-shot LLM obligation profile, compiles a certificate-carrying harness, converts verifier diagnostics into a profile patch if coverage fails, and recompiles with the exact compiler. Unlike the router repair baselines, repair decisions are routed back through the declared-registry compiler rather than emitted as an unconstrained module set.

Table 6 changes the claim. On GapBench test800, one-shot workflow generation is weaker than GapHarness

LLM and the LLM Tool Router. Verifier-repair and ReAct-style selection reach 1.00 harness success after feedback, but they do so without certificates. GapHarness-Repair also reaches 1.00 while preserving a compiler certificate and using one LLM profile call, with an average 0.11 feedback/recompile steps. On HarnessChallenge-200, feedback-assisted baselines reach 1.00; GapHarness-Repair does so with certificates and 0.30 feedback/recompile steps on average. We therefore do not claim universal raw-success dominance over feedback loops. The claim is that obligation-first compilation separates profile inference from exact certified compilation, and that feedback can be incorporated by verifier-guided recompilation without losing registry-relative certificates.

Table 6: Diagnostic-feedback strategy baselines. HS is verifier coverage; cost/excess use declared module costs. Verifier-repair, ReAct, and GapHarness-Repair receive verifier diagnostics after failed routes; they should be interpreted as feedback-assisted upper bounds.

Dataset	System	N	HS	Decl. cost	Excess	Over	Under	Wrong	Calls	Steps	Cert.
GapBench test800	GapHarness LLM	800	0.89	3.59	0.37	0.14	0.09	0.01	1.00	1.00	yes
GapBench test800	Registry-guarded GH	800	0.94	3.98	0.38	0.15	0.03	0.01	1.00	1.00	yes
GapBench test800	LLM Tool Router	800	0.80	3.51	0.13	0.12	0.20	0.17	1.00	1.00	no
GapBench test800	Workflow Generator	800	0.77	3.37	0.11	0.10	0.23	0.18	1.00	1.00	no
GapBench test800	Verifier-Repair Router	800	1.00	3.85	0.16	0.14	0.00	0.00	1.20	0.20	no
GapBench test800	ReAct Module Selector	800	1.00	3.90	0.21	0.20	0.00	0.00	1.08	1.08	no
GapBench test800	GapHarness-Repair	800	1.00	3.96	0.28	0.15	0.00	0.00	1.00	0.11	yes
HarnessChallenge-200	GapHarness LLM	200	0.69	3.92	0.96	0.05	0.15	0.11	1.00	1.00	yes
HarnessChallenge-200	Registry-guarded GH	200	0.59	4.82	1.86	0.05	0.15	0.11	1.00	1.00	yes
HarnessChallenge-200	LLM Tool Router	200	0.65	2.60	0.04	0.01	0.35	0.28	1.00	1.00	no
HarnessChallenge-200	Workflow Generator	200	0.83	3.31	0.10	0.03	0.17	0.17	1.00	1.00	no
HarnessChallenge-200	Verifier-Repair Router	200	1.00	3.62	0.15	0.04	0.00	0.00	1.41	0.41	no
HarnessChallenge-200	ReAct Module Selector	200	1.00	3.63	0.15	0.04	0.00	0.00	1.22	1.22	no
HarnessChallenge-200	GapHarness-Repair	200	1.00	3.69	0.21	0.05	0.00	0.00	1.00	0.30	yes

6.6 Feedback leakage and certificate utility

The diagnostic-feedback baselines above are intentionally strong and potentially leaky: after failure they receive coverage-checker diagnostics. To make that dependence visible, we replay repair under three feedback levels. Weak feedback exposes only pass/fail; medium feedback exposes missing obligation families; strong feedback exposes missing capabilities/status and is the upper-bound setting used in Table 6. This replay is deterministic and does not replace the LLM baseline runs; it measures how much of the repair advantage comes from the amount of verifier information.

Table 7: Feedback-level replay. Weak feedback is pass/fail only, medium reveals missing obligation families, and strong reveals missing capabilities/status. Cost/excess use declared module costs. Strong feedback is an upper bound. Cert. reports whether the repaired route has a compiler certificate.

Dataset	Feedback	System	N	HS	Decl. cost	Excess	Over	Cert.
GapBench test800	weak	GapHarness-Repair replay	800	0.89	3.59	0.37	0.14	yes
GapBench test800	weak	Router/ReAct repair replay	800	1.00	5.87	2.18	0.31	no
GapBench test800	medium	GapHarness-Repair replay	800	0.91	3.62	0.37	0.14	yes
GapBench test800	medium	Router/ReAct repair replay	800	0.93	3.78	0.16	0.14	no
GapBench test800	strong	GapHarness-Repair replay	800	1.00	3.94	0.25	0.14	yes
GapBench test800	strong	Router/ReAct repair replay	800	1.00	3.82	0.13	0.12	no
HarnessChallenge-200	weak	GapHarness-Repair replay	200	0.69	3.92	0.96	0.05	yes
HarnessChallenge-200	weak	Router/ReAct repair replay	200	1.00	6.56	3.08	0.36	no
HarnessChallenge-200	medium	GapHarness-Repair replay	200	0.79	4.39	0.98	0.05	yes
HarnessChallenge-200	medium	Router/ReAct repair replay	200	0.79	2.83	0.04	0.01	no
HarnessChallenge-200	strong	GapHarness-Repair replay	200	1.00	3.64	0.17	0.04	yes
HarnessChallenge-200	strong	Router/ReAct repair replay	200	1.00	3.52	0.04	0.01	no

The weak-feedback replay is deliberately blunt: router-style repair can recover coverage by falling back toward a broad harness, but does so with much higher excess cost and over-harnessing. Medium feedback is more realistic than strong capability/status hints and reduces the gap between strategy families. Strong feedback should be read as an upper bound, not as evidence that the baseline can infer obligations unaided.

We also quantify certificate utility with deterministic proxies and a prepared human-audit packet. The proxy

estimates whether a debugger can identify the failure cause from route metadata, counts redundant modules, and scores whether a missing cause is localized. It does not report completed human timing; the audit packet and review sheet are included for the human follow-up. The key observation is narrower: certificates reduce proxy-estimated diagnostic work in both held-out and targeted settings while preserving explicit coverage/minimality evidence.

Table 8: Certificate utility proxy. This is a deterministic proxy plus an audit packet, not a completed human timing study.

Dataset	System	N	HS	Cert.	Audit proxy	Debug work	Missing cause localized
GapBench test800	LLM Tool Router	800	0.80	0.00	0.94	4.13	0.70
GapBench test800	Workflow Generator	800	0.77	0.00	0.93	4.15	0.70
GapBench test800	Verifier-Repair Router	800	1.00	0.00	1.00	3.78	1.00
GapBench test800	ReAct Module Selector	800	1.00	0.00	1.00	3.80	1.00
GapBench test800	GapHarness LLM	800	0.89	1.00	1.00	2.45	1.00
GapBench test800	GapHarness-Repair	800	1.00	1.00	1.00	2.40	1.00
HarnessChallenge-200	LLM Tool Router	200	0.65	0.00	0.89	4.16	0.70
HarnessChallenge-200	Workflow Generator	200	0.83	0.00	0.95	4.03	0.70
HarnessChallenge-200	Verifier-Repair Router	200	1.00	0.00	1.00	3.77	1.00
HarnessChallenge-200	GapHarness LLM	200	0.69	1.00	1.00	2.89	1.00
HarnessChallenge-200	GapHarness-Repair	200	1.00	1.00	1.00	2.38	1.00

6.7 Cost-scheme sensitivity, status confusion, and profiler errors

Because registry-relative minimality depends on the declared cost function, we run a cost-scheme sensitivity check with simple latency, token/API, and risk proxies, then replay major strategies under alternative cost schemes. Table 9 shows that the qualitative pattern is stable: GapHarness LLM remains higher coverage than direct workflow/router baselines; repair baselines achieve 1.00 coverage but pay additional excess; and certificates remain the distinguishing property of GapHarness variants. These cost schemes are proxies, not measurements of live provider billing.

Table 9: Cost sensitivity on GapBench test800. Costs are declared/proxy schemes over the same selected modules.

Scheme	System	N	HS	Proxy cost	Excess	Over
declared	LLM Tool Router	800	0.80	3.51	0.13	0.12
declared	GapHarness LLM	800	0.89	3.59	0.37	0.16
declared	Verifier-Repair Router	800	1.00	3.85	0.16	0.14
declared	GapHarness-Repair	800	1.00	3.96	0.28	0.15
uniform	LLM Tool Router	800	0.80	2.02	0.13	0.12
uniform	GapHarness LLM	800	0.89	2.10	0.26	0.16
uniform	Verifier-Repair Router	800	1.00	2.22	0.16	0.14
uniform	GapHarness-Repair	800	1.00	2.28	0.21	0.15
latency proxy	LLM Tool Router	800	0.80	4.48	0.13	0.12
latency proxy	GapHarness LLM	800	0.89	4.73	0.47	0.16
risk weighted	LLM Tool Router	800	0.80	4.77	0.23	0.12
risk weighted	GapHarness LLM	800	0.89	4.96	0.56	0.16
token/API proxy	LLM Tool Router	800	0.80	4.16	0.13	0.12
token/API proxy	GapHarness LLM	800	0.89	4.35	0.46	0.16

Status confusion and error taxonomy make the profiler bottleneck explicit. On GapBench test800, the LLM profiler’s largest status error is supported tasks predicted unsupported; the post-hoc guard reduces this from 56 to 12 but leaves clarify behavior mostly unchanged. On HarnessChallenge-200, the same guard fails: it predicts supported for all 50 unsupported boundary rows, while the LLM Tool Router returns unsupported for all unsupported boundary rows but under-covers supported rows. This explains why the guard is calibration, not a general solution.

Table 10: Selected status confusion counts. Rates are normalized within each expected-status group.

Dataset	System	Expected	Predicted	N	Rate
GapBench test800	GapHarness LLM	supported	unsupported	56	0.07
GapBench test800	Registry-guarded GH	supported	unsupported	12	0.02
GapBench test800	GapHarness LLM	unsupported	unsupported	24	1.00
GapBench test800	GapHarness LLM	clarify	clarify	6	0.25
GapBench test800	LLM Tool Router	clarify	clarify	22	0.92
HarnessChallenge-200	GapHarness LLM	unsupported	supported	30	0.60
HarnessChallenge-200	Registry-guarded GH	unsupported	supported	50	1.00
HarnessChallenge-200	LLM Tool Router	unsupported	unsupported	50	1.00

The profiler error taxonomy separates false unsupported, missing multi-obligation, verification/control boundary confusion, coarse capability, and dependency errors. GapBench failures are dominated by false unsupported predictions; HarnessChallenge failures are dominated by dependency misses and unsupported false positives. This provides a concrete target for future profiler training and prevents the aggregate harness-success number from hiding boundary-specific mistakes.

Table 11: LLM profiler error taxonomy. Rates are among failed rows for each dataset. Categories are not mutually exclusive.

Dataset	Error category	N	Rate
GapBench test800	false unsupported	56	0.63
GapBench test800	clarify/unsupported confusion	18	0.20
GapBench test800	dependency missed	15	0.17
HarnessChallenge-200	dependency missed	31	0.51
HarnessChallenge-200	unsupported false positive	30	0.49
HarnessChallenge-200	missing multi-obligation	23	0.38
HarnessChallenge-200	verification/control confusion	19	0.31

6.8 Stress tests and negative controls

Registry perturbation removes one key module at a time and runs only relevant subsets. Removing `python_executor`, `source_span_checker`, `permission_gate`, `sandbox_file_editor`, `web_retrieval`, or `contract_verifier` reduces relevant-subset harness success from 1.00 to 0.00 and yields missing capabilities matching the removed affordance. This verifies that GapHarness does not hallucinate support beyond the declared registry. In perturbation rows, wrong-harnessing means verifier-visible missing coverage under the original supported profile, not an incorrect final answer.

Gold label permutation corrupts 200 supported profiles while the verifier still checks original labels. Correct profiles yield 1.00 harness success and 0.00 cost delta. Permuted profiles reduce harness success to 0.17 and raise under-harnessing to 0.83 and wrong-harnessing to 0.79. This is an anti-circularity stress test, not a realistic label-noise model.

Negative controls include pure-language prompts and tool-bait prompts that mention tools while explicitly asking not to use them. GapHarness gold, GapHarness LLM, registry-guarded GapHarness, and the LLM Tool Router all avoid over-harnessing these categories. Heuristic Tool Router and Difficulty Router over-harness tool-bait at 0.51, and Always-full over-harnesses both categories at 1.00.

Table 12: Stress-test summary. Perturbation rows report perturbed harness success; label permutation reports verifier outcomes against original labels.

Test	Condition	N	HS	Under	Wrong
Registry perturbation	remove execution module	60	0.00	1.00	1.00
Registry perturbation	remove source checker	60	0.00	1.00	1.00
Registry perturbation	remove permission gate	60	0.00	1.00	1.00
Registry perturbation	remove sandbox editor	60	0.00	1.00	1.00
Registry perturbation	remove web retrieval	60	0.00	1.00	1.00
Registry perturbation	remove contract verifier	60	0.00	1.00	1.00
Label permutation	correct gold	200	1.00	0.00	0.00
Label permutation	permuted labels	200	0.17	0.83	0.79

6.9 HarnessChallenge-200 targeted diagnostic

HarnessChallenge-200 is designed to be adversarial rather than naturally sampled. It contains minimal pairs, hard tool-bait, sandbox versus real side-effect boundaries, registry absence cases, evidence traps, and real-source paraphrases. The diagnostic asks whether labels and declared registry affordances matter. Under gold profiles, GapHarness and the oracle minimal harness reach 1.00 harness success and zero cost delta. Always-full succeeds on supported rows but fails unsupported boundary rows and over-harnesses heavily. Direct, heuristic routers, and direct LLM module routing under-cover many multi-obligation rows.

The LLM Tool Router reaches 0.65 harness success on HarnessChallenge-200, while GapHarness with LLM-inferred obligations reaches 0.69. The GapBench-calibrated registry guard drops to 0.59 on this harder diagnostic set because it does not solve broad registry-absence and side-effect-boundary profiling errors. We report this as a boundary result, not as a positive-only ablation: the guard is useful on GapBench test800 but not a general replacement for better obligation profiling.

The diagnostic-feedback baselines in Table 6 show why this boundary matters. Workflow generation reaches 0.83, and feedback-driven repair/ReAct variants reach 1.00 on this targeted diagnostic. GapHarness-Repair also reaches 1.00, but routes the feedback through profile patching and exact recompilation, preserving a certificate. This does not invalidate the one-shot compiler result; it shows that verifier feedback can repair many profiling mistakes. The paper’s claim is therefore about separating obligation inference from certificate-carrying compilation under a declared registry, not about beating every iterative agent policy on raw coverage.

Table 13: HarnessChallenge-200 targeted diagnostic. This benchmark is constructed to stress obligation semantics and registry boundaries; it is not a natural-frequency benchmark. Cost columns use declared module costs.

System	N	HS	Decl. cost	Delta	Excess	Over	Under/Wrong
Direct	200	0.28	0.00	-3.48	0.00	0.00	0.47 / 0.00
Tool Router	200	0.29	2.60	-0.87	1.21	0.29	0.46 / 0.31
Difficulty Router	200	0.33	4.64	1.17	2.85	0.33	0.42 / 0.21
Always-full	200	0.75	16.00	12.53	12.53	0.75	0.00 / 0.00
LLM Tool Router	200	0.65	2.60	-0.88	0.04	0.01	0.35 / 0.28
GapHarness gold	200	1.00	3.48	0.00	0.00	0.00	0.00 / 0.00
GapHarness LLM	200	0.69	3.92	0.45	0.96	0.05	0.15 / 0.11
Registry-guarded GH	200	0.59	4.82	1.34	1.86	0.05	0.15 / 0.11
Oracle minimal	200	1.00	3.48	0.00	0.00	0.00	0.00 / 0.00

Bootstrap 95% CIs for HarnessChallenge harness success are [0.630, 0.755] for GapHarness LLM, [0.580, 0.710] for the LLM Tool Router, and [1.000, 1.000] for gold-profile GapHarness.

6.10 SWE-HarnessExec-20 executable trace validation

SWE-HarnessExec-20 tests whether compiled harnesses can drive an actual sandboxed software-maintenance loop. Each case begins with a local Python file and a failing pytest test. The runner applies a provided patch, reruns pytest, stores the diff/log artifacts, and verifies that the expected pre-failure and post-success

trace occurred. This is not model patch generation and not SWE-bench pass@1; it is a small execution-level check of the harness loop.

GapHarness gold and oracle minimal both achieve 1.00 coverage success and 1.00 trace success at declared cost 12.00. Always-full also executes successfully but costs 16.00 under the declared scheme and over-harnesses every row. Direct, Tool Router, and Difficulty Router stop before the trace because they lack required declared modules such as `sandbox_file_editor`, `permission_gate`, `state_store`, `execution_log_checker`, or `contract_verifier`.

We additionally run the LLM-inferred pipeline and diagnostic-feedback strategies on the same 20 executable fixtures. GapHarness LLM, registry-guarded GapHarness, the LLM Tool Router, Workflow Generator, Verifier-Repair Router, ReAct Module Selector, and GapHarness-Repair all reach 1.00 coverage and trace success at declared cost 12.00. This is a useful boundary result: for homogeneous execution-heavy tasks with obvious required modules, direct module routing and agentic strategies can match obligation-first profiling. GapHarness’ advantage is therefore concentrated in mixed, boundary-sensitive, unsupported, and tool-bait cases, and in producing a certificate when a profile is available.

Table 14: SWE-HarnessExec-20 executable trace validation. Trace success requires failing pytest before the provided patch, passing pytest after the patch, and verifier-visible trace artifacts. Cost uses declared module costs.

System	N	Coverage HS	Trace HS	Decl. cost	Pre-fail	Post-pass	Calls	Cert.
Direct	20	0.00	0.00	0.00	0.00	0.00	0.00	no
Tool Router	20	0.00	0.00	4.00	0.00	0.00	0.00	no
Difficulty Router	20	0.00	0.00	6.00	0.00	0.00	0.00	no
Always-full	20	1.00	1.00	16.00	1.00	1.00	0.00	no
GapHarness gold	20	1.00	1.00	12.00	1.00	1.00	0.00	yes
Oracle minimal	20	1.00	1.00	12.00	1.00	1.00	0.00	yes
GapHarness LLM	20	1.00	1.00	12.00	1.00	1.00	1.00	yes
Registry-guarded GH	20	1.00	1.00	12.00	1.00	1.00	1.00	yes
LLM Tool Router	20	1.00	1.00	12.00	1.00	1.00	1.00	no
Workflow Generator	20	1.00	1.00	12.00	1.00	1.00	1.00	no
Verifier-Repair Router	20	1.00	1.00	12.00	1.00	1.00	1.00	no
ReAct Module Selector	20	1.00	1.00	12.00	1.00	1.00	1.00	no
GapHarness-Repair	20	1.00	1.00	12.00	1.00	1.00	1.00	yes

To reduce dependence on the initial 20-fixture executable sample, we additionally scale the deterministic provided-patch runner to SWE-HarnessExec-50. The added cases cover common software-maintenance micro-failures such as parsing defaults, normalization, grouping, interval checks, pagination, TTL boundaries, currency formatting, and JSON-lines parsing. This scale-up still has the same boundary: it verifies supplied patches through local pytest traces and does not generate repairs. Table 15 shows that GapHarness gold and oracle minimal continue to execute every trace successfully at declared cost 12.00, Always-full succeeds with higher declared cost, and direct/router baselines stop before execution because the required execution/action/state/control/verification affordances are absent.

Table 15: SWE-HarnessExec-50 provided-patch scale-up. Trace success requires failing pytest before the supplied patch and passing pytest after the patch. Cost uses declared module costs.

System	N	Coverage HS	Trace HS	Decl. cost	Pre-fail	Post-pass	Missing-module
Direct	50	0.00	0.00	0.00	0.00	0.00	1.00
Tool Router	50	0.00	0.00	4.00	0.00	0.00	1.00
Difficulty Router	50	0.00	0.00	6.00	0.00	0.00	1.00
Always-full	50	1.00	1.00	16.00	1.00	1.00	0.00
GapHarness gold	50	1.00	1.00	12.00	1.00	1.00	0.00
Oracle minimal	50	1.00	1.00	12.00	1.00	1.00	0.00

6.11 Secondary adversarial audit

We add a secondary LLM audit over a stratified GapBench-100 sample. The sample is stratified by category. The auditor receives query text, obligation definitions, and registry descriptions, but not gold labels or system outputs. We report this as an adversarial consistency check, not as human inter-annotator agreement or an independent human audit.

Table 16: Secondary LLM audit on GapBench-100.

N	Obl. exact	Obl. micro-F1	Cap. micro-F1	Status agree	Harness exact
100	0.65	0.878	0.814	0.87	0.75

Disagreements concentrate in ambiguous action targets, `sandbox_action` versus `contract_check`, the boundary of Verification, and Control versus Action dependencies. This is useful pressure: it shows the taxonomy has real semantic edges, and those edges should be documented rather than treated as invisible decoration.

6.12 External-validity and boundary diagnostics

GAIA-Transfer gold reaches 1.00 harness success under project-owner-audited transfer labels, showing that the compiler can process transfer-style obligation profiles. This is not GAIA answer solving. GAIA registry-guarded reaches only 0.56 harness success and high over/under rates, which exposes a limitation: the current registry and guard are tuned for GapBench sandbox/action boundaries and do not cover broader multimodal, file, evidence, and state-boundary demands.

GapBench-Natural now contains 200 project-owner-audited naturalized rows. It reaches 1.00 harness success under gold profiles, but it remains GapBench-derived and is therefore weaker external-validity evidence than real-source tasks.

SWE-Obligation-50 is derived from public SWE-bench Lite task descriptions and test metadata. Under the original project-owner-audited source view, GapHarness gold reaches 1.00 harness success at declared cost 12.00 and zero cost delta, while Direct and deterministic Tool Router under-cover all rows. Always-full succeeds but over-harnesses with declared cost 16.00. For LLM calls, all LLM-based systems receive the same shortened diagnostic view, with repository-specific failure details compressed but the task intent, expected modification type, and test/trace hints preserved. The shortened view is used only to avoid provider content filters and long issue text; it is not used for gold-label adjudication or as a replacement for the original source view. On that shared view, GapHarness LLM reaches 1.00 harness success at declared cost 12.80, and the LLM Tool Router reaches 1.00 at declared cost 12.00. This supports an obligation-transfer claim for real software-maintenance task descriptions, not a SWE-bench solving or pass@1 claim.

Table 17: External-validity and boundary diagnostics. These artifacts are not primary performance evidence and do not claim GAIA, Terminal-Bench, or SWE-bench solving.

Artifact	Identity	N	HS	Decl. cost	Boundary
GAIA-Transfer gold	transfer labels	200	1.00	1.48	not GAIA solving
GAIA guarded	limitation diagnostic	200	0.56	5.56	multimodal/evidence gap
GapBench-Natural	project-owner-audited naturalization	200	1.00	2.83	still GapBench-derived
SWE-Obligation-50	real-source obligation transfer	50	1.00	12.00	not patch solving
SWE LLM-safe	LLM diagnostic view	50	1.00	12.80	shortened view; source labels original

Terminal-Bench-obligation50 remains an appendix scaffold derived from public terminal-style task instructions and is not a Terminal-Bench solving result.

7. Evaluation Scope

The main metric is harness coverage, not final answer correctness. GapBench is controlled, which is a strength for isolating the compiler/verifier path and a limitation for broad ecological validity. Compiler replay and scaling evaluate exact optimizer behavior, not end-user task utility. HarnessChallenge-200 is intentionally adversarial and should not be interpreted as natural task frequency. RealBoundary-100 is a fresh author-seeded holdout, but it is review-pending. Naturalistic-Holdout v0.1 is an independent 200-row candidate package from public GitHub issues, but it is not scored until two-annotator labeling, agreement analysis, and adjudication are complete. SWE-HarnessExec-20 and SWE-HarnessExec-50 add execution-level trace evidence, but with provided patches and small sandbox fixtures rather than open-ended model repair. Registry-guarded calibration is explicitly post-hoc. Transfer runs are external-validity and boundary diagnostics. Stress tests are anti-circularity checks, not realistic corruption models. These constraints are not hidden threats to the claim; they define the layer evaluated in this paper. GapHarness provides an auditable layer for declared-registry runtime support compilation, not a complete proof of agent competence.

8. Limitations

The default executor is deterministic and sandbox/mock only. The SWE-HarnessExec runner performs real local file edits and pytest execution inside generated sandbox fixture directories, but it does not perform irreversible file edits outside the sandbox, real API calls, payments, emails, deployments, or production changes.

Minimality is registry-relative. Changing module granularity, costs, capabilities, or dependencies changes the minimal harness. The optimized compiler remains exponential in the worst case; dominance pruning and branch-and-bound improve practical behavior for the declared registries studied here but do not make harness compilation polynomial. Exactness is intended for small-to-medium declared runtime registries, not arbitrary tool universes. For large registries, GapHarness should be preceded by retrieval/candidate narrowing or replaced by an approximation policy, with the certificate reflecting that narrower candidate set.

LLM profiling remains the main practical bottleneck. The LLM profiler outperforms direct and router baselines on the main held-out GapBench test but still under-harnesses some multi-obligation tasks. HarnessChallenge-200 makes this limitation sharper: registry absence and real side-effect boundaries remain difficult for the current LLM profiler, and the GapBench-calibrated registry guard can hurt on targeted diagnostics. Verifier-repair and ReAct-style baselines show that diagnostic feedback can recover coverage on these controlled diagnostics, so raw harness success alone should not be read as the central advantage. GapHarness-Repair uses the same kind of feedback to patch profiles and recompile, preserving certificates, but it is still a feedback-assisted upper-bound variant rather than a one-shot profiler result. In SWE-HarnessExec-20, direct LLM module routing and diagnostic-feedback strategies match GapHarness LLM because the required modules are obvious and homogeneous. Registry guarding fixes one systematic false-positive pattern and should not be generalized beyond that boundary without fresh holdout evidence.

GapBench-Natural is project-owner-audited, but it is still a naturalization of GapBench source rows rather than an independent external benchmark. Naturalistic-Holdout v0.1 is independent of GapBench, but its annotation fields remain candidate/review fields until two annotators complete independent passes, agreement is reported, and disagreements are adjudicated. RealBoundary-100 is fresh relative to the registry guard but is still author-seeded and review-pending. SWE-Obligation-50 uses real SWE-bench Lite task descriptions, but it is obligation-transfer only: it does not check out repositories, generate patches, execute SWE-bench tests, or report pass@1. SWE-HarnessExec-20 and SWE-HarnessExec-50 run real local tests, but their patches are provided and their repositories are small generated fixtures. Terminal-Bench-obligation50 remains appendix material until its prompts and labels receive independent audit and task-level validators.

9. Conclusion

GapHarness reframes API-only agent harnessing as a compiler problem over explicit obligations and declared runtime affordances. Under gold labels, exact optimizing compilation gives a simple relative-minimality guarantee and matches the oracle minimal harness on GapBench-1000. Dominance pruning, branch-and-bound search, and certificate-carrying outputs make this compilation more auditable without changing frozen harness outputs. With LLM-inferred profiles, GapHarness improves over direct, heuristic router, and LLM Tool Router baselines on the main held-out benchmark. Diagnostic-feedback baselines show the remaining tradeoff: repair/ReAct policies can reach perfect verifier coverage on controlled diagnostics, but they do not provide registry-relative minimality certificates. GapHarness-Repair shows the natural extension: verifier diagnostics can be converted into profile constraints and recompiled, reaching repair-level coverage while preserving certificates. Feedback-level replay, certificate-utility proxies, cost-scheme sensitivity, status confusion, profiler error taxonomy, registry perturbation, label permutation, negative controls, targeted diagnostics, executable traces, and a secondary audit reduce the likelihood that observed gains arise from keyword routing, unconditional tool use, or label-insensitive compilation.

The resulting claim is intentionally bounded but concrete: for API-only agents whose runtime affordances are declared as a finite registry, obligation-first compilation can produce minimal, certificate-backed, verifier-visible harnesses and fail explicitly when required affordances are absent. Its advantage is best understood as certificate-preserving, registry-constrained support selection: compared with direct routing it improves coverage on the main held-out benchmark, and compared with iterative repair it preserves auditable minimality evidence rather than claiming lower declared cost or universal raw-success dominance. The next step is not to overclaim open-world solving, but to expand independently audited naturalistic benchmarks, add larger executable repository traces, and improve obligation profiling under the same declared-registry semantics.

Appendix A. Negative-Control Table

Table 18: Negative controls. All GapHarness variants and the LLM Tool Router avoid tool-bait over-harnessing; heuristic routers over-harness tool-bait. Cost uses declared module costs.

Category	System	N	HS	Decl. cost	Over
pure-language	Direct	100	1.00	0.00	0.00
pure-language	Tool Router	100	1.00	0.00	0.00
pure-language	LLM Tool Router	100	1.00	0.00	0.00
pure-language	Difficulty Router	100	1.00	0.00	0.00
pure-language	Always-full	100	1.00	16.00	1.00
pure-language	GapHarness gold	100	1.00	0.00	0.00
pure-language	GapHarness LLM	100	1.00	0.00	0.00
pure-language	Registry-guarded GH	100	1.00	0.00	0.00
tool-bait	Direct	100	1.00	0.00	0.00
tool-bait	Tool Router	100	1.00	1.26	0.51
tool-bait	LLM Tool Router	100	1.00	0.00	0.00
tool-bait	Difficulty Router	100	1.00	1.22	0.51
tool-bait	Always-full	100	1.00	16.00	1.00
tool-bait	GapHarness gold	100	1.00	0.00	0.00
tool-bait	GapHarness LLM	100	1.00	0.00	0.00
tool-bait	Registry-guarded GH	100	1.00	0.00	0.00

Appendix B. Related-Work Comparison

Table 19: Positioning against related tool, workflow, planning, and harness work. Entries summarize the paper-scope comparison rather than exhaustive capabilities of each system.

Work	Harness/registry model	Obligation/minimality evidence	Real execution?	Difference from GapHarness
ReAct / Toolformer	Tool-use policies over an available tool set	No exact minimality or registry-relative certificate	mixed	Studies tool-using behavior; GapHarness compiles declared runtime support from obligations.
Gorilla / ToolLLM / MetaTool	API/tool catalog selection	Tool-level necessity, not obligation-first exact compilation	mixed	Focuses on API selection; GapHarness separates obligation inference from support compilation.
AutoFlow / AFlow / WorFBench	Workflow/DAG generation or evaluation	Workflow objectives rather than finite affordance minimality	benchmark-dependent	GapHarness targets minimal runtime coverage under declared affordances.
AutoHarness / NLAH / Harness-Bench	Code or language harnesses, or harness-effect benchmarks	No finite declared-registry minimality certificate	yes	These study harness synthesis/specification/effects; GapHarness selects support modules with certificates.
Set cover / service composition	Finite services/features under constraints	Optimization ancestry; certificates are not the LLM-agent artifact	no	GapHarness applies this lineage to auditable LLM runtime support.
Runtime verification / assurance cases	Runtime claims, policies, or safety arguments	Checks behavior against explicit claims	yes	GapHarness compiles and certifies the support set before execution.
GapHarness	Finite declared runtime-affordance registry	Obligation-first exact selection with registry-relative certificate	sandbox traces	Certificate-carrying runtime harness compilation for API-only LLM agents.

Appendix C. Case Studies

Tool-bait negative control. Query: “The word ‘search’ appears here only as an example. Do not use tools, browsing, code execution, files, or external state. Answer in one sentence using only this prompt.” The audited profile has no obligations and an empty oracle harness. GapHarness and the LLM Tool Router return direct-answer support, while Always-full necessarily over-harnesses and heuristic routers often select unnecessary modules because tool words appear in the prompt.

Unsupported real-world action. Query: “Send a real Slack announcement to the entire company now.” The audited profile requires Action, Control, and Verification with `real_world_side_effect`, but the registry intentionally contains only sandbox/mock actions. GapHarness returns unsupported with a missing-affordance certificate. A system that silently maps this to `sandbox_file_editor` or `permission_gate` alone is wrong because the declared registry does not support the requested real-world side effect.

Multi-obligation executable trace. Query family: “Inspect the workspace fixture, run pytest, apply the provided patch only in the sandbox, rerun pytest, and verify the trace.” The profile requires Observation, Execution, State, Action, Control, and Verification. The minimal harness includes workspace reading, Python execution, execution-log checking, state storage, sandbox editing, permission gating, and contract verification. Direct and simple routers under-cover, Always-full succeeds with excess cost, and GapHarness produces the minimal declared support set plus a certificate.

References

- Benavides, David, Sergio Segura, and Antonio Ruiz-Cortés. 2010. “Automated Analysis of Feature Models 20 Years Later: A Literature Review.” *Information Systems* 35 (6): 615–36. <https://doi.org/10.1016/j.is.2010.01.001>.
- Chvátal, Vasek. 1979. “A Greedy Heuristic for the Set-Covering Problem.” *Mathematics of Operations Research* 4 (3): 233–35. <https://doi.org/10.1287/moor.4.3.233>.
- Huang, Yue, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, et al. 2023. “MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use.” *arXiv Preprint arXiv:2310.03128*. <https://arxiv.org/abs/2310.03128>.

- Kang, Kyo C., Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. 1990. "Feature-Oriented Domain Analysis (FODA) Feasibility Study." CMU/SEI-90-TR-021. Software Engineering Institute, Carnegie Mellon University. <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=11231>.
- Karp, Richard M. 1972. "Reducibility Among Combinatorial Problems." In *Complexity of Computer Computations*, edited by Raymond E. Miller and James W. Thatcher, 85–103. New York: Plenum Press. https://doi.org/10.1007/978-1-4684-2001-2_9.
- Kelly, Tim, and Rob Weaver. 2004. "The Goal Structuring Notation: A Safety Argument Notation." In *Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases*. <https://www-users.york.ac.uk/~tpk/gsn.pdf>.
- Kephart, Jeffrey O., and David M. Chess. 2003. "The Vision of Autonomic Computing." *Computer* 36 (1): 41–50. <https://doi.org/10.1109/MC.2003.1160055>.
- Leucker, Martin, and Christian Schallhart. 2009. "A Brief Account of Runtime Verification." *The Journal of Logic and Algebraic Programming* 78 (5): 293–303. <https://doi.org/10.1016/j.jlap.2008.08.004>.
- Li, Zelong, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. "AutoFlow: Automated Workflow Generation for Large Language Model Agents." *arXiv Preprint arXiv:2407.12821*. <https://arxiv.org/abs/2407.12821>.
- Liu, Xiao, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, et al. 2024. "AgentBench: Evaluating LLMs as Agents." In *International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Lou, Xinghua, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. "AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness." *arXiv Preprint arXiv:2603.03329*. <https://arxiv.org/abs/2603.03329>.
- Merrill, Mike A., Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, et al. 2026. "Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces." *arXiv Preprint arXiv:2601.11868*. <https://arxiv.org/abs/2601.11868>.
- Mialon, Grégoire, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. "GAIA: A Benchmark for General AI Assistants." In *International Conference on Learning Representations*. <https://arxiv.org/abs/2311.12983>.
- Pan, Linyue, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 2026. "Natural-Language Agent Harnesses." *arXiv Preprint arXiv:2603.25723*. <https://arxiv.org/abs/2603.25723>.
- Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. "Gorilla: Large Language Model Connected with Massive APIs." *arXiv Preprint arXiv:2305.15334*. <https://arxiv.org/abs/2305.15334>.
- Qiao, Shuofei, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2025. "Benchmarking Agentic Workflow Generation." In *International Conference on Learning Representations*. <https://arxiv.org/abs/2410.07869>.
- Qin, Yujia, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. 2024. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs." In *International Conference on Learning Representations*. <https://openreview.net/forum?id=dHng2O0Jjr>.
- Rao, Jinghai, and Xiaomeng Su. 2005. "A Survey of Automated Web Service Composition Methods." *Lecture Notes in Computer Science* 3387: 43–54. https://doi.org/10.1007/978-3-540-30581-1_5.
- Saltzer, Jerome H., and Michael D. Schroeder. 1975. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63 (9): 1278–308. <https://doi.org/10.1109/PROC.1975.9939>.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. "Toolformer: Language Models Can Teach Themselves to Use Tools." In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2302.04761>.
- Wang, Zhenting, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, et al. 2025. "MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers." *arXiv Preprint arXiv:2508.20453*. <https://arxiv.org/abs/2508.20453>.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." In *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.03629>.
- Yao, Yilun, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, et al. 2026. "Harness-Bench: Measuring Harness Effects Across Models in Realistic Agent Workflows." *arXiv*

Preprint arXiv:2605.27922. <https://arxiv.org/abs/2605.27922>.
Yu, Peijie, Wei Liu, Yifan Yang, Jinjian Li, Zelong Zhang, Xiao Feng, and Feng Zhang. 2026. “Benchmarking LLM Tool-Use in the Wild.” *arXiv Preprint arXiv:2604.06185*. <https://arxiv.org/abs/2604.06185>.
Zhang, Jiayi, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, et al. 2025. “AFlow: Automating Agentic Workflow Generation.” In *International Conference on Learning Representations*. <https://openreview.net/forum?id=z5uVAKwmjf>.